ELEC 475 Lab 1

Post Lab

Nivesh Gandhimathinathan (20415227)

Fateen Elhaam Bin Fisal (20430786)

1. Model

The model is a four layer fully connected autoencoder, implemented in model.py as the class autoencoder "MLP4Layer". It takes a 28x28 FashionMNIST image and flattenes it into a 1 dimensional image with 784 input vectors and compresses it though two encoder layers into a low dimensional bottle neck, then reconstruct the original image though two decoder layers as shown in Figure 1

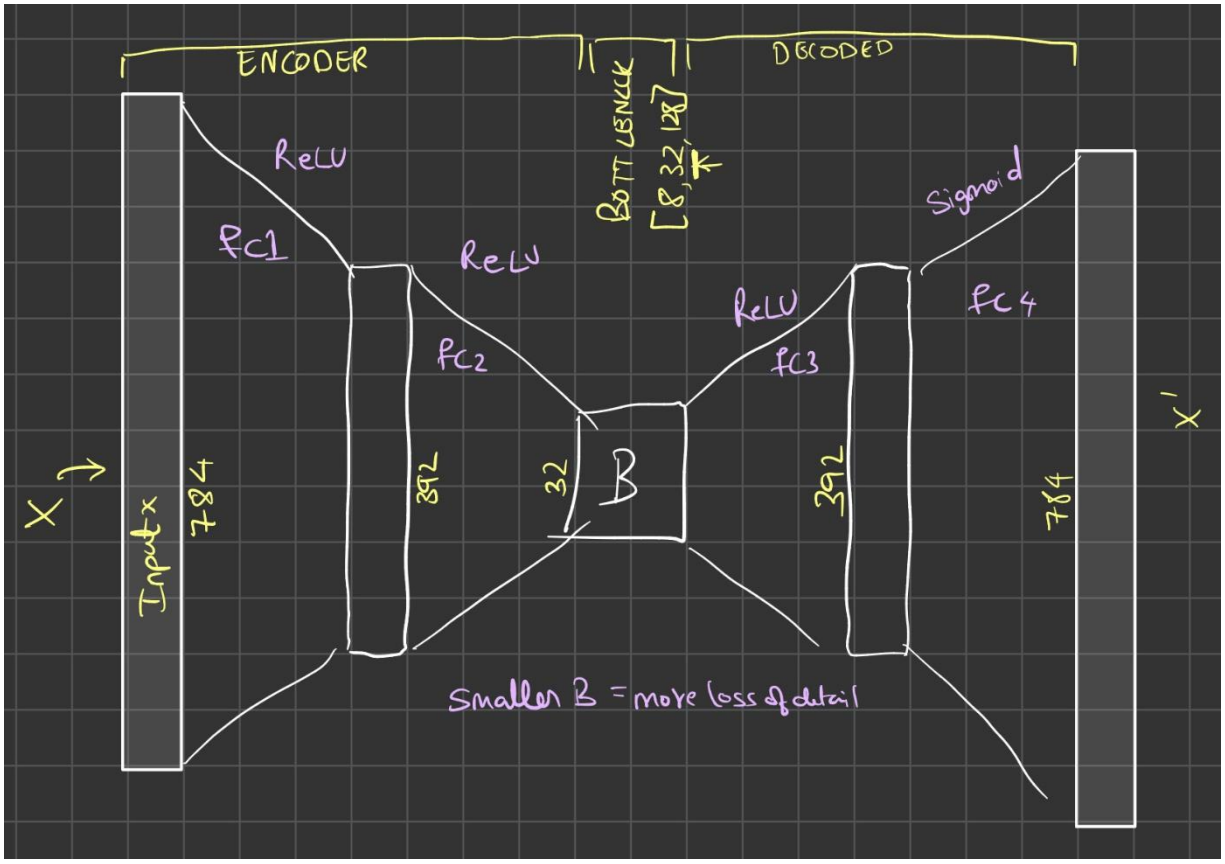


Figure 1: Auto Encoder

Here N2 = N_input//2 =392 with B as the bottle neck and the values for B evaluated are 8, 32 and 128.

The encoder (fc1 and fc2) maps the 784 dimension image down to the size of the bottle neck B. The decoder (fc3, fc4) then maps the Bottleneck image back to a 784 dimensional reconstruction. The ReLU (max(0,z)) is used after the first three layers to be the standard non linear activation for hidden layers. However, for the last layer, Sigmoid is used to constrain the reconstructed pixel values to the range [0,1], which matches the input pixel range.

For B = 32, fc1: 784 x 392 + 392 =307720

Fc2: 392 x32 +32 = 12576

Fc3: 32x392 +392 = 12936

Fc4: 392x784 +784 = 308112

This gives us a total parameter count of 641344 which was noticed in the output from train.py for B=32.

Similar process is followed for bottlenecks 8 and 128.

2.Training

The model was trained on the full FashionMNIST training set of 60,000 grayscale 28x28 images, with no validation split. Each image was converted to a tensor using **transforms.ToTensor(),** which scales pixel values to the range [0, 1], and was then flattened into a 784-element vector before being passed to the network. The training data was shuffled every epoch and loaded in batches of 256.

Before training, the weights of every linear layer were initialized with Xavier uniform initialization and all biases were set to 0.01. The loss function was the mean squared error (**`nn.MSELoss`**) between the reconstructed image and the original input image. The network was optimized with Adam, using a learning rate of 1e-3 and a weight decay of 1e-5. A `ReduceLROnPlateau` learning rate scheduler was used in `min` mode with PyTorch's default settings (factor 0.1, patience 10), and was stepped once per epoch using the summed training loss for that epoch.

Training ran for 30 epochs on [CPU/GPU]. The model weights were saved after every epoch, and the average training loss per epoch was plotted to produce the loss curve. No random

seed was set, so repeated runs produce slightly different loss values. The model was trained with the following command:

```
python train.py -z 32 -e 30 -b 256 -s MLP.32.pth -p loss.MLP.32.png
```

The models with bottleneck sizes of 8 and 128 were trained with identical settings, changing only the bottleneck size (`-z`), save file (`-s`) and plot file (`-p`) arguments. The loss curves for all three models are shown in Figure 8,Figure 10 and Figure 12.

3. Outputs

3.1 Testing



Figure 2: Reconstruction of pullover index 1

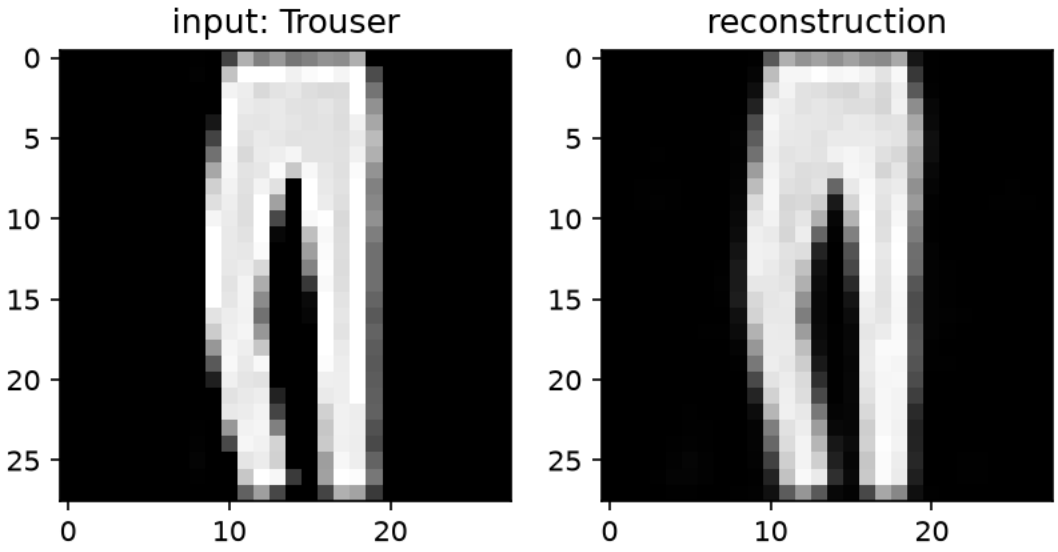


Figure 3: Reconstruction of trouser index 2

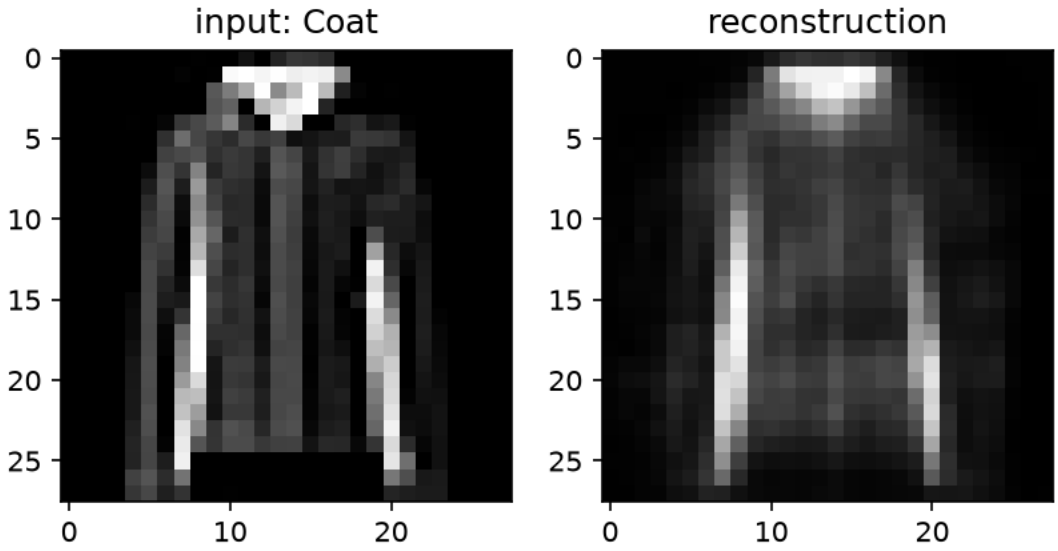


Figure 4: Reconstruction of coat index 6

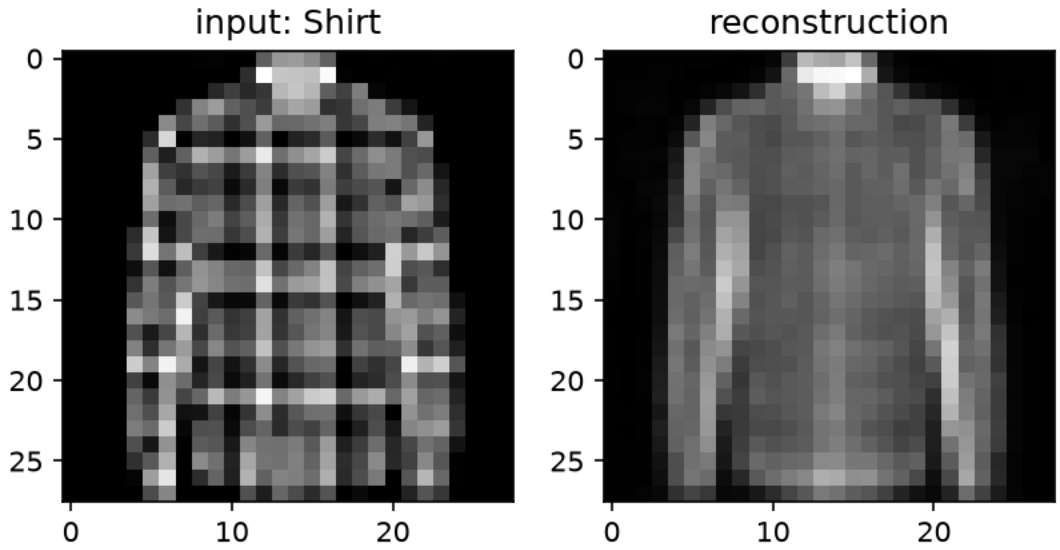


Figure 5: Reconstruction of shirt index 7

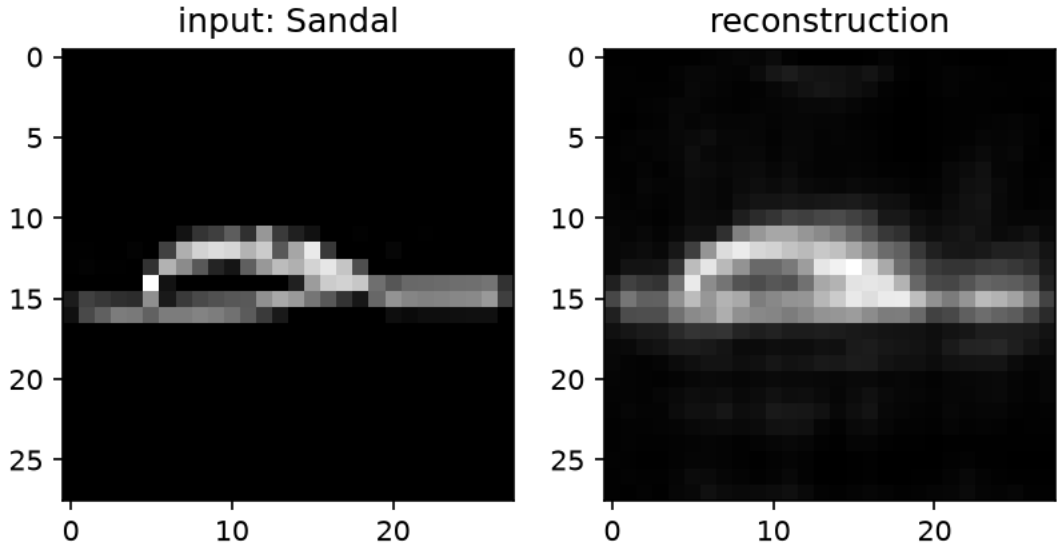


Figure 6: Reconstruction of sandal index 8

3.2 Denoise

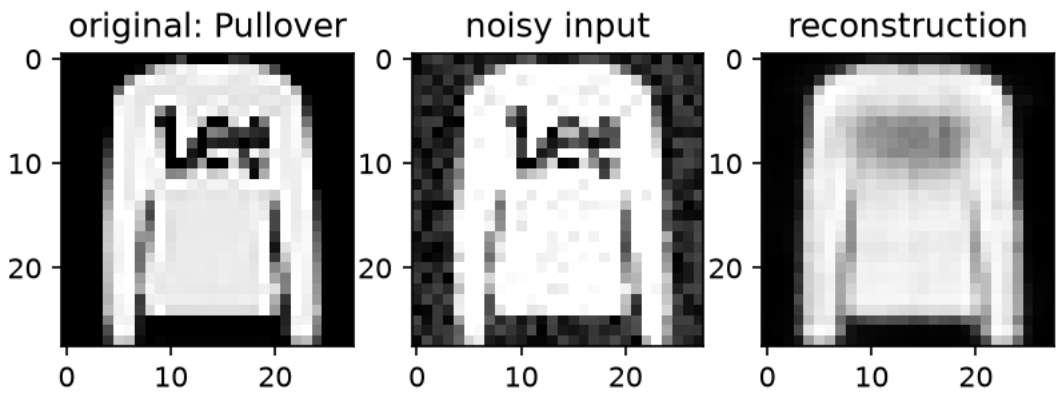


Figure 7: Image reconstruction with denoise.py

3.3 Bottleneck size

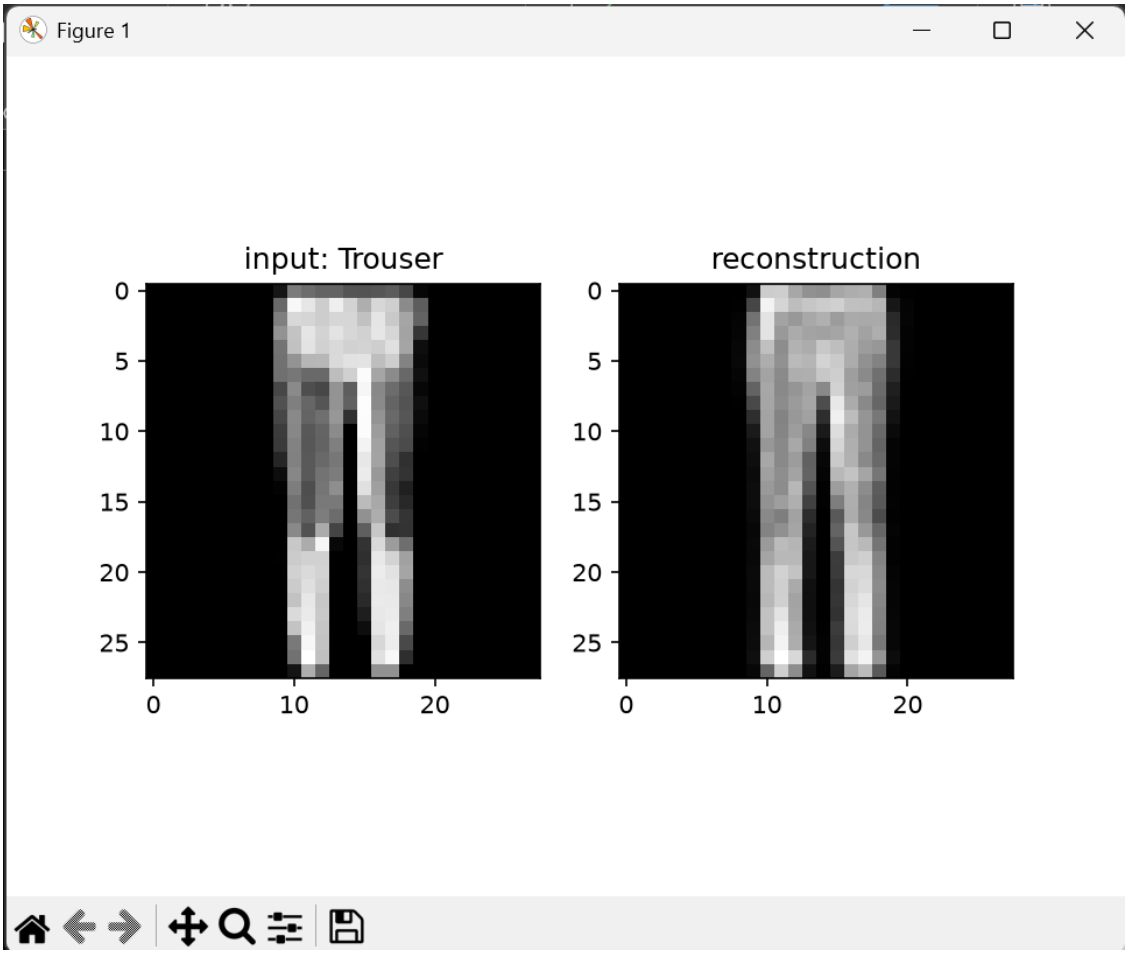


Figure 8: Reconstruction with bottleneck 8

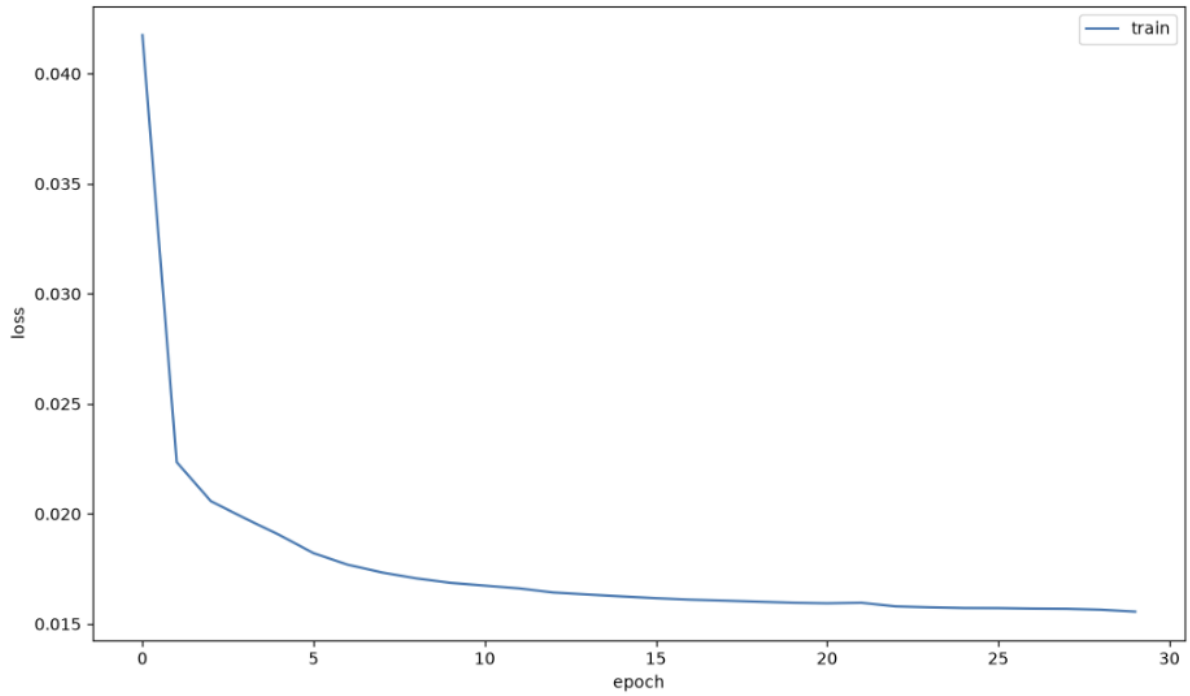


Figure 9: Loss for bottleneck 8

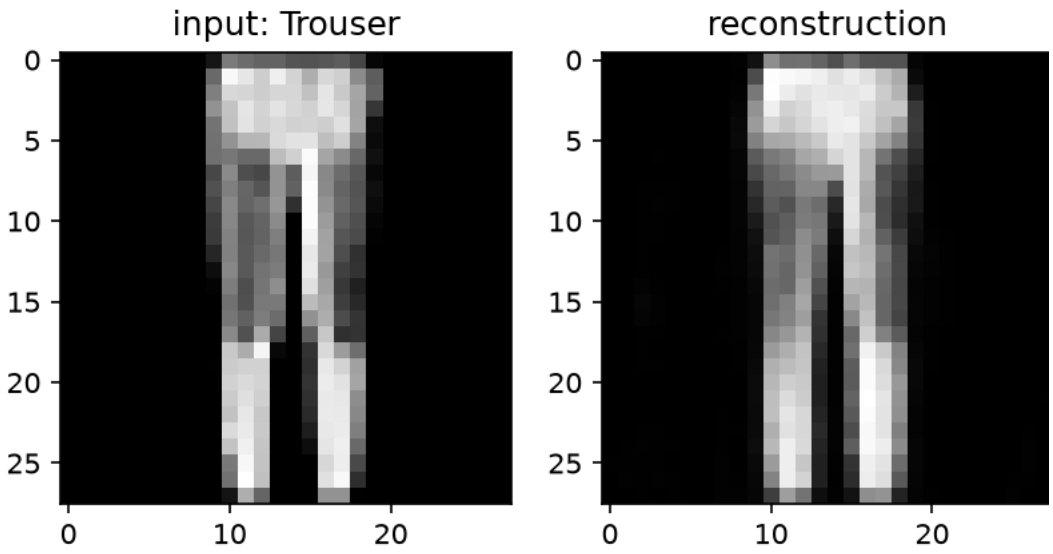


Figure 10: Reconstruction with bottleneck 32

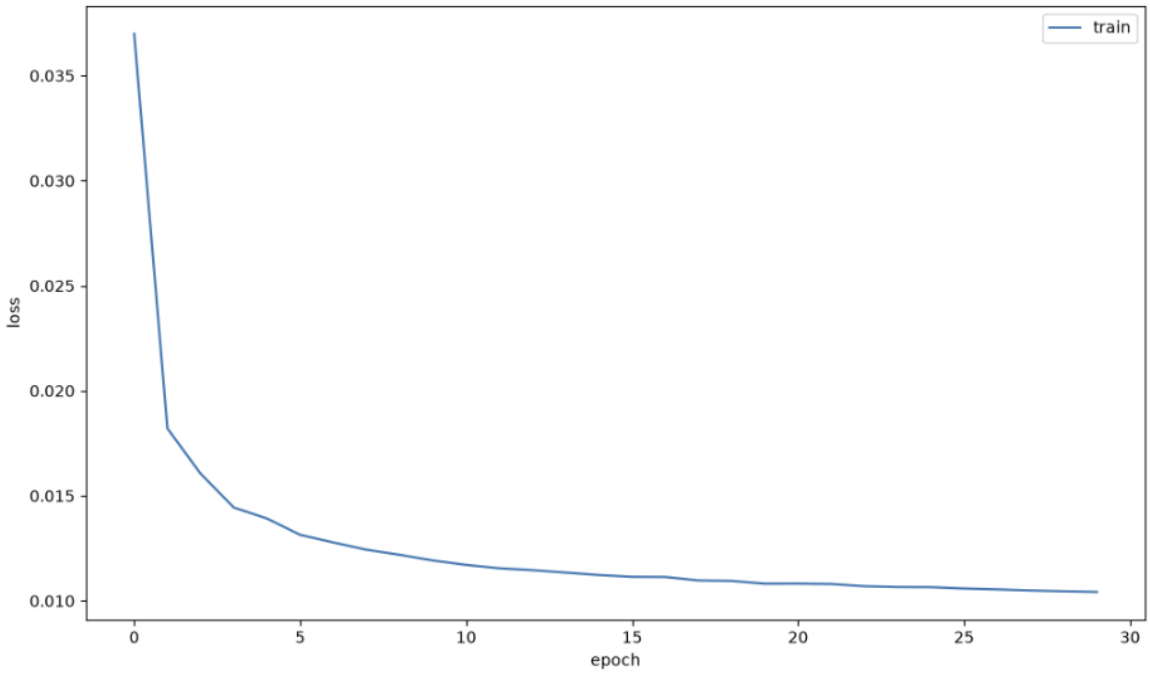


Figure 11: Reconstruction with bottleneck 32

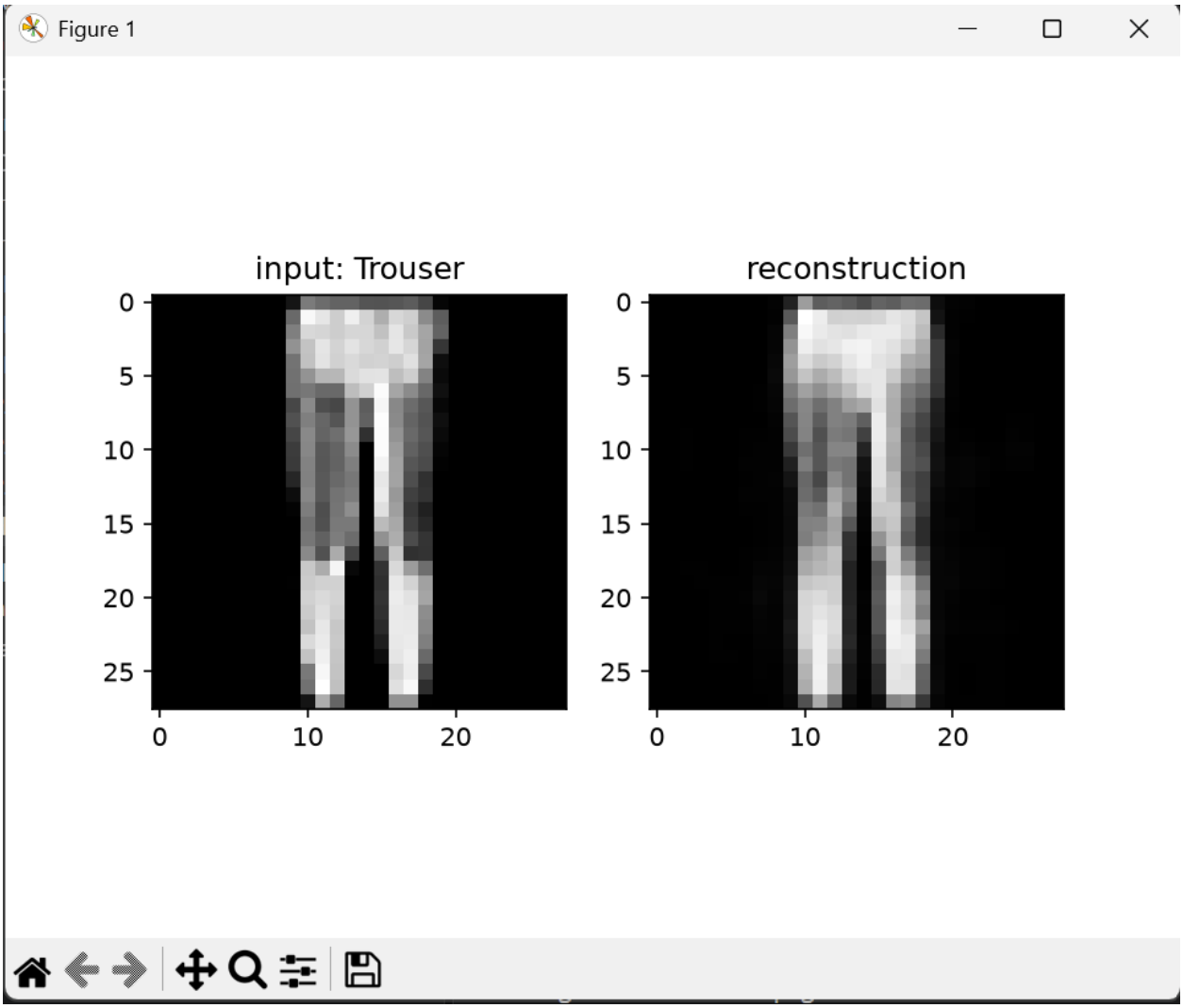


Figure 12: Reconstruction with bottleneck 128

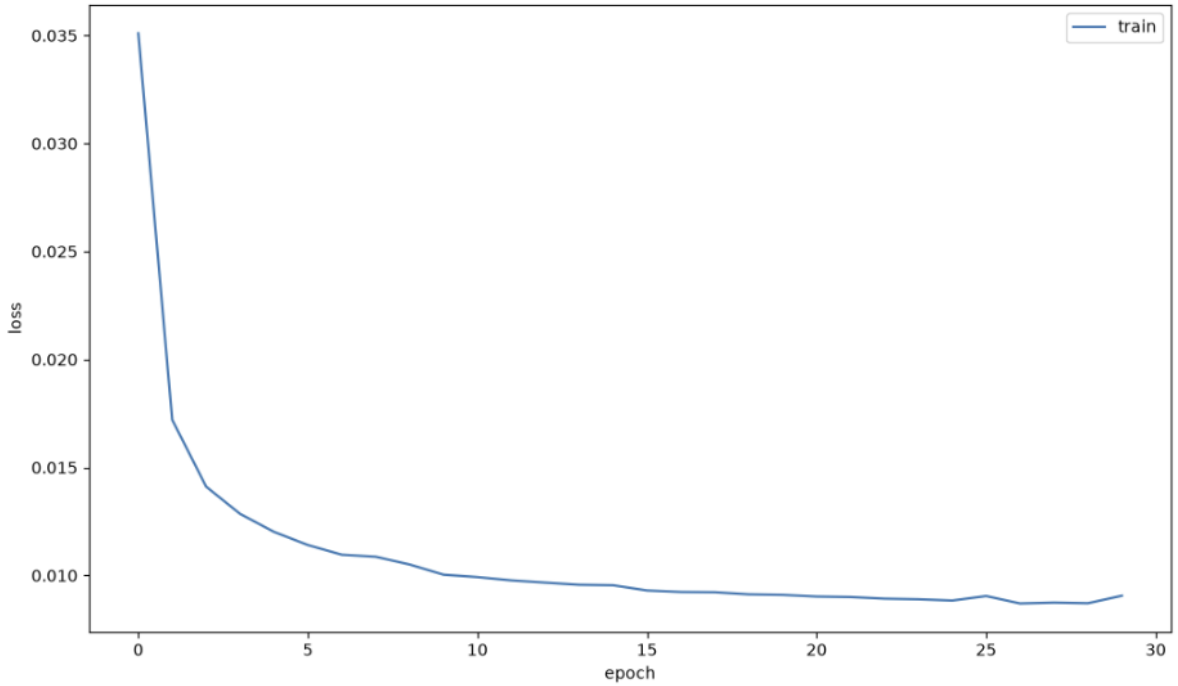


Figure 13: Loss for bottleneck 128

3.4 Bottleneck interpolation

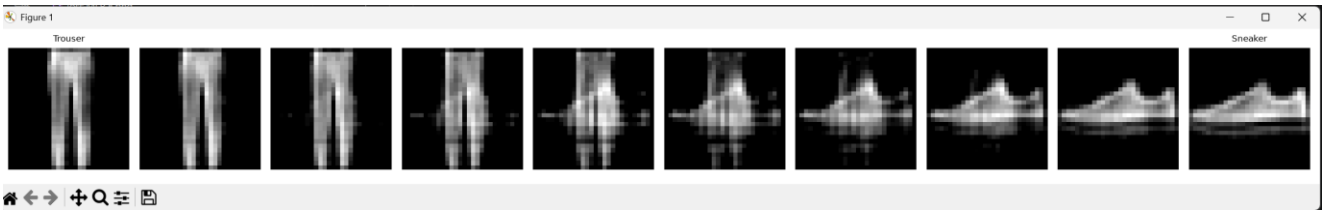


Figure 14: Interpolation between trouser and sneaker

4.Analysis

a) Based on Figure 2 the information which survives is the overall shape/silhouette of the object. The details do not transfer well and as the edges are smoothed out.

b) The bottleneck only has 32 numbers, so it keeps the overall shape of the clothing and has no room to store random noise. The decoder was only trained on clean images, so it rebuilds a clean version. The downside is that small details are lost too, like the "Lee" logo in Figure 7.

c) A bigger bottleneck gives a better reconstruction because the model has more numbers to describe each image. With a bottleneck of 8 Figure 8, only the rough shape survives and the images are very blurry, while 32 as seen in Figure 10 gives a clear shape but still loses small details. With 128 as seen in Figure 12, the reconstructions are the sharpest and closest to the originals.

d) The images change gradually from the trouser to the sneaker, with the legs fading out as the sneaker shape grows in Figure 14. The middle images look like a mixture of both items, with the trouser legs and the sneaker shape visible at the same time, though blurry. This suggests the bottleneck is smooth and continuous, so codes in between two images decode to something in between them.